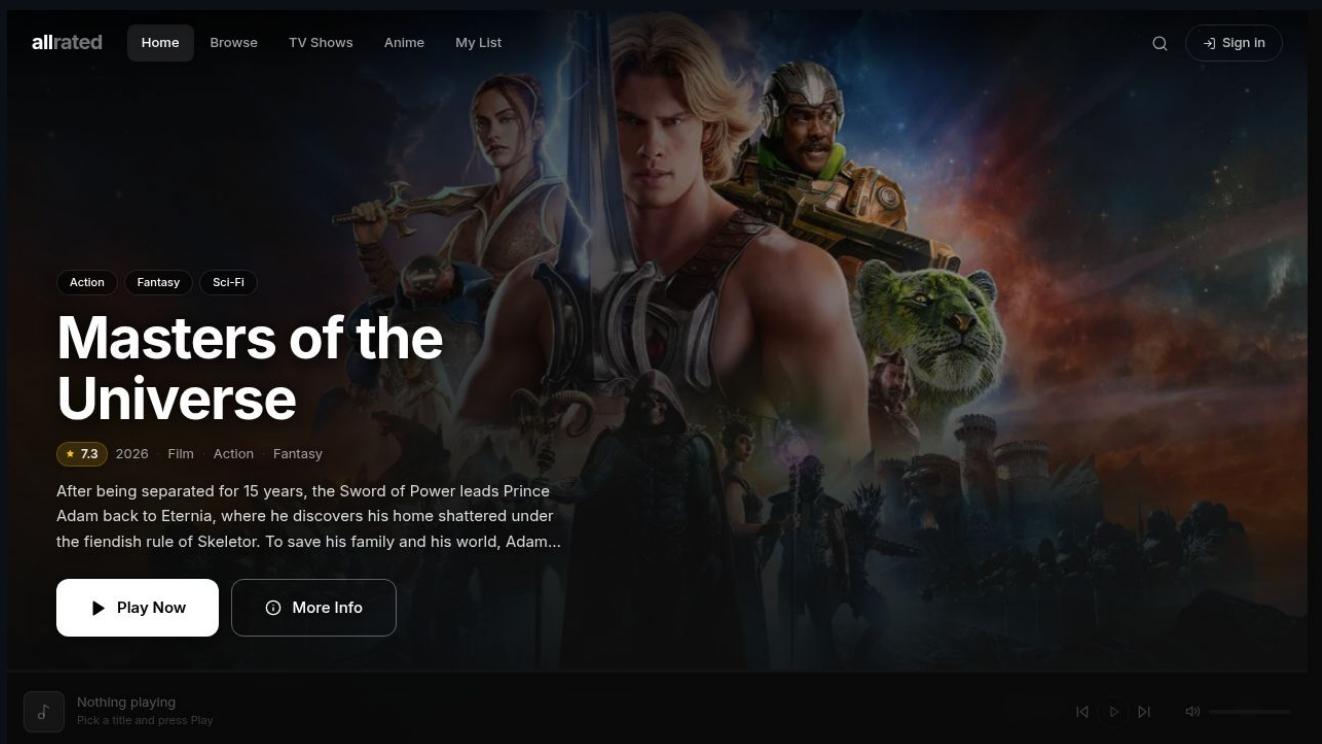


Allrated

A practical summary of the product, codebase, confirmed risks, and the next visual evolution of the movie, TV, and anime discovery platform.

EXECUTIVE READ

Allrated already has a strong cinematic foundation: live TMDB discovery, title detail pages, authentication, watchlist, history, multiple playback providers, and a Railway-backed API. The biggest opportunity is not a rewrite. It is tightening the release path, simplifying interaction semantics, and adding a more intentional visual system around discovery and playback.



Current Replit preview snapshot. The homepage is rendering live TMDB content with the cinematic hero treatment.

Scope: code and UI audit of the current workspace snapshot; recommendations are separated from confirmed implementation findings.

Contents



Use this as both a product brief and an implementation backlog.

#	Section	Purpose
01	Product snapshot	What Allrated is and who it serves
02	Architecture & code map	How the frontend, Railway API, database, auth, and providers connect
03	What is already strong	Working foundations worth preserving
04	Confirmed bugs & risks	Issues observed from the code and current deployment workflow
05	UI/UX audit	Where the experience can become clearer, faster, and more premium
06	Animation direction	A restrained motion system for a cinematic streaming product
07	Future product features	Ideas ranked by user value and implementation effort
08	Prioritized roadmap	What to fix first, next, and later
09	Definition of done	Acceptance checklist for the next release

01 / Product snapshot

Allrated is positioned as a content-first discovery and playback companion, not only a catalog.

Core promise: help someone decide what to watch, save it, rate it in a distinctive four-tier system, and resume it across devices.

Area	Current reality
Product type	Movie, TV, anime, and sports discovery platform with playback integrations.
Primary audience	People who browse across genres and providers, want a quick decision, and expect a Netflix-like cinematic interface.
Signature feature	Four-tier ratings: Skip, Timepass, Go for it, and Perfection — owned by Allrated rather than copied from TMDB.
Content source	TMDB supplies live homepage metadata; the backend stores resolved titles, user data, ratings, watchlist, and progress.
Current visual language	Near-black cinematic surfaces, white typography, glass navigation, large backdrops, soft borders, and restrained amber/maroon accents.
Current release split	Replit runs the React/Vite frontend. Railway runs the Express/Prisma API and PostgreSQL-backed data layer. GitHub main is the deployment bridge.

The homepage already communicates the right emotional territory: a full-bleed hero, high-contrast title treatment, immediate Play Now / More Info actions, and horizontal content rows. The product should now become more disciplined around hierarchy, feedback, and production consistency.

02 / Architecture & code map



The current architecture is serviceable and should be improved incrementally rather than migrated.

Layer	Implementation	Key locations
Frontend	React 18 + Vite + Tailwind + Framer Motion + Lucide	client/src/App.tsx, client/src/pages, client/src/components
Navigation	Route-level lazy loading, shared TopNav, mobile bottom navigation	client/src/App.tsx, client/src/components/layout
Discovery	Direct TMDB client service with five-minute memory cache and normalized title shape	client/src/services/tmdb.ts, client/src/pages/Home.tsx
API client	Fetch wrapper with Supabase session token, request dedupe, and selected TTL caches	client/src/lib/api.ts
Authentication	Supabase Auth client-side session + Railway token verification + user upsert	client/src/lib/auth.tsx, server/src/middleware/auth.ts
Backend	Express API with title, watchlist, history, provider, sports, and admin routes	server/src/server.ts, server/src/routes
Database	PostgreSQL via Prisma; User, Title, Rating, WatchlistItem, WatchProgress, Platform, KV	server/prisma/schema.prisma
Deployment	Railway/Nixpacks builds client then server; production server serves client/dist	server/src/server.ts, client/nixpacks.toml, railway configs

Measured surface: 21 page modules, 34 reusable components, 13 backend route modules, and roughly 65 route handlers were found in the current source tree. That is enough surface area to justify a small shared design-system layer and automated smoke tests.

Production flow: [edit client/](#) → verify locally → commit to GitHub main → Railway rebuilds the production service. Backend changes must reach GitHub/Railway; frontend-only changes are visible in Replit immediately but are not live on Railway until released.

03 / What is already strong



Preserve these foundations while improving the rough edges.

Cinematic thesis

HeroSection uses full-bleed imagery, readable scrim, a controlled Ken Burns effect, auto-advance, swipe support, trailer mute state, and clear primary actions.

Useful content model

The normalized TMDB shape keeps movie and TV cards consistent while retaining tmdbId/mediaType for backend resolution.

Good performance instincts

Route-level lazy loading, image lazy loading, five-minute TMDB caching, one-minute API hot-cache entries, request dedupe, and GPU-friendly animation are all good choices.

Real user state

Watchlist, ratings, watch progress, episode context, and history are persisted through the backend rather than simulated in the UI.

Safety baseline

Bearer-token auth, server-side role checks, rate limiting, Prisma relations with cascade behavior, and centralized error handling provide a useful baseline.

Responsive structure

The app has a desktop top navigation, mobile menu, fixed mobile bottom navigation, safe-area padding, touch actions, and responsive hero behavior.

KEEP

Do not replace the client/server split, Supabase Auth, Prisma, or the existing visual language just to make the code look newer. The highest return is in consistency and release discipline.

04 / Confirmed bugs & risks

Severity is based on user impact, release impact, and likelihood of causing confusing behavior.

Severity	Finding	Evidence / impact	Fix
P0	Replit and Railway can show different frontend builds.	The current smaller-card edits are local/uncommitted while Railway deploys GitHub main. Users can see a correct Replit preview and an older Railway UI.	Commit and push frontend changes; add a release checklist and a visible build/version marker.
P1	ContentCard has nested interactive controls.	The outer card is a role=button div containing Play, Add, and Info buttons. This is invalid interaction semantics and can confuse keyboard and assistive technology users.	Make the card a non-interactive article with a dedicated link/button target, or move card actions outside the outer interactive element.
P1	Client-side TMDB key is embedded in the browser bundle.	VITE_TMDB_API_KEY is intentionally used directly by the homepage. It is not a secret once shipped and can be abused for quota exhaustion.	Proxy discovery through Railway or add a server cache/rate policy. Keep only public configuration in the client.
P1	Homepage load fans out into many TMDB requests.	Six initial content requests plus up to five trailer requests can run near the same time. Slow or rate-limited TMDB responses degrade the first paint.	Prioritize hero + first row, defer trailers, use server aggregation, and add request cancellation.
P1	Auth/config initialization fails silently.	Auth catches /api/config failure and only flips isLoading off. The user gets no clear explanation if Railway config or Supabase setup is unavailable.	Render a recoverable configuration error with Retry and a support-safe diagnostic ID.
P1	Service-worker cache can preserve stale production UI.	The product has client/public/sw.js and a release split; stale assets can make a Railway update appear not to deploy.	Version the cache, clean old caches on activate, call skipWaiting/clientsClaim carefully, and surface app build version.
P2	API request cleanup may create unhandled rejection noise.	api.ts calls request.finally(...) without consuming the promise returned by finally. A rejected request can produce a secondary unhandled rejection.	Use void request.finally(...).catch(() => {}) or move cleanup into a single async wrapper.
P2	WatchProgress stores one row per user/title.	The unique userId + titleId constraint keeps only the latest episode context. It cannot represent per-episode history or rewatch state.	Add an episode progress table or a history event model if episode-level analytics/history is a goal.
P2	Mobile menu has too many destinations.	TopNav's mobile menu includes nine destinations while the bottom nav includes five. This creates competing navigation systems.	Choose one primary mobile navigation model and make secondary destinations available through Profile/More.
P2	Accessibility state is incomplete in filters and carousels.	Home tabs do not expose aria-pressed; horizontal rows need stronger region/scroll affordance semantics.	Add labelled regions, aria-pressed, keyboard scroll controls, and visible focus tests.

Important distinction: legacy CSS such as unused provider-card and season-tabs rules is maintenance debt, not an immediate user-facing bug. Remove it during a focused cleanup pass after current behavior is covered by screenshots or tests.

05 / UI/UX audit



The product feels cinematic today; the next step is making every interaction feel equally intentional.

Visual hierarchy

- Keep the hero as the thesis, but reduce the amount of competing metadata below it. The title, rating, one-line context, and two actions should win the first scan.
- Use a clearer difference between discovery rows: Trending can be image-led, Top 10 can be rank-led, and personalized Continue Watching can be progress-led.
- Keep small cards small. The current card system is now being reduced to 92px mobile / 124px desktop in the workspace; the release process must push that change to Railway so production matches preview.
- Use one accent behavior for primary action, one for rating, and one for alerts. Avoid introducing extra neon colors that compete with poster art.

Interaction clarity

- Every action should produce an immediate state: loading label, pressed state, toast, or progress update. Especially important for source switching and Play actions.
- Replace hover-only meaning with touch-safe states. Desktop hover can reveal controls, but mobile should expose a predictable tap target and never rely on hover.
- Give the user a clear reason when a title cannot resolve, a provider fails, an episode list is empty, or a TMDB request is unavailable.
- Keep the bottom navigation to five items and treat the full-screen menu as a secondary directory, not a competing primary nav.

Accessibility and trust

- Fix nested card interactivity first. It affects keyboard flow, screen readers, and the ability to predict what Enter/Space will activate.
- Audit muted gray text against the dark background. The UI uses many white-alpha values; convert the most important labels to named accessible tokens.
- Add focus-visible styles to every custom button, menu item, carousel control, source option, and episode control.
- Use alt text that describes the title, not just an empty decorative image, on poster cards where the poster communicates title identity.

06 / Animation direction



Motion should make Allrated feel alive without making browsing slower or noisy.

Moment	Recommended motion	Guardrail
Hero transition	Crossfade the image and stagger genre → title → metadata → actions over 180–320ms.	Do not animate layout height; respect reduced motion.
Card reveal	Fade/translate rows into view once, with 30–50ms stagger between cards.	Use IntersectionObserver; never animate every scroll frame.
Poster hover	Keep the existing 1.08 scale, but add a soft poster-color glow and a short play-button spring.	Cap the scale so adjacent cards do not jump or clip.
Source selector	Chevron rotation + 160ms menu slide/fade; active source gets a clear check/playing state.	Keep the menu keyboard navigable and outside overflow clipping.
Continue Watching	Animate progress fill only when progress changes; show a subtle resume pulse after returning to the page.	Avoid autoplaying decorative pulses.
Loading	Use skeleton shimmer for the first paint and a small inline spinner for retries.	Reserve image dimensions to prevent layout shift.
Feedback	Use lightweight bottom toast for saved, removed, rating saved, and provider failure.	Do not use blocking alerts for routine actions.

MOTION RULE

One signature moment is enough: a calm cinematic hero transition. Everything else should be fast, quiet, and functional.

Motion tokens to standardize: 160ms for micro feedback, 240ms for menus, 320ms for content reveals, and 500ms for cinematic image crossfades. Use the existing cubic-bezier token and add a reduced-motion branch to every new animation.

07 / Future product features

Prioritized ideas that improve retention and differentiation instead of only adding more screens.

Priority	Feature	User value	Implementation shape
Now	Release/version indicator	Makes Replit vs Railway drift obvious and builds trust during updates.	Expose build SHA/date from a generated client constant; show only in Profile/About or an admin diagnostic.
Now	Reliable error + retry system	Users understand whether a provider, API, or network failed.	Shared request error type, retry button, toast, and route-level error states.
Now	Better onboarding	New users understand four-tier ratings, My List, and resume playback immediately.	Three-step lightweight first-session coach marks; never block browsing.
Next	Personalized home rows	Turns passive browsing into a reason to return.	Use watchlist status, ratings, history, and genres to rank rows; keep TMDB fallback.
Next	Episode-level progress	Makes TV/anime resume behavior accurate across devices.	Separate EpisodeProgress model or normalized history events.
Next	Provider health and fallback	Reduces dead-end Play actions.	Measure provider success/failure, show availability state, and automatically suggest the next source.
Next	Watch-party / shareable title cards	Creates organic discovery loops.	Share route with OG metadata, poster, rating, and deep link; later add synchronized rooms.
Later	Taste profile	Makes recommendations feel owned by Allrated.	Explicit likes/dislikes, favorite genres, language preferences, and recommendation explanation.
Later	Installable PWA polish	Improves return visits on mobile.	Offline shell, cache versioning, install prompt, update notice, and watchlist/history resilience.
Later	Moderation and admin analytics	Protects catalog quality as the user base grows.	Provider uptime, failed resolutions, report queue, sync status, and audit log.

08 / Prioritized roadmap



A realistic sequence for shipping quality without destabilizing the current stack.

Phase	Timebox	Work	Exit criteria
0. Release hygiene	1 day	Commit/push current card sizing; confirm Railway rebuild; add app build marker; fix service-worker cache versioning.	Replit and Railway show the same card dimensions and build ID after a hard refresh.
1. Interaction correctness	2–4 days	Remove nested interactive card semantics; add aria states; unify mobile navigation; fix API finally cleanup.	Keyboard-only smoke test passes for nav, cards, search, source picker, episodes, and playback actions.
2. Perceived quality	3–5 days	Add toasts, provider loading/error states, section-level retry, hero transition polish, and empty-state copy.	Every async action has visible loading, success, empty, and failure feedback.
3. Performance	3–5 days	Defer trailer calls, prioritize hero/first row, add abort controllers, audit image sizes, and test mobile 4G.	First useful content appears quickly; no avoidable layout shift or request storm.
4. Product retention	1–2 weeks	Personalized rows, episode progress model, recommendation explanations, and shareable title links.	Returning users see relevant content and resume the correct episode.
5. Visual signature	1–2 weeks	Accent refinement, taste profile UI, ambient hero lighting, richer genre tiles, and polished PWA update flow.	The product has a memorable identity beyond “dark streaming clone” while staying fast.

Recommended first commit: release the small-card change, then immediately follow it with the interaction-correctness pass. A beautiful card that behaves like a confusing button is still a quality problem.

Suggested branch structure: [release/card-sizing](#) → [fix/card-a11y](#) → [polish/feedback-states](#) → [perf/home-loading](#).

09 / Definition of done

Use this list before calling the next release complete.

Area	Release must prove
Release	The GitHub commit contains the intended frontend changes; Railway build succeeds; Replit and Railway show the same build marker.
Visual	Cards have the intended width on 375px, 768px, 1024px, and 1440px; poster art does not clip; Top 10 ranks remain visible.
Navigation	Desktop nav, mobile menu, bottom nav, browser back, deep links, and Escape behavior are predictable.
Playback	Play, source selection, episode selection, loading, provider failure, and resume progress all provide visible feedback.
Accessibility	No nested interactive controls; focus-visible is visible; buttons have labels; tabs expose selected state; contrast is checked.
Performance	Images reserve aspect ratio; first row is lazy; trailer calls are deferred; no avoidable duplicate requests.
Reliability	TMDB failure, Railway API failure, Supabase config failure, offline mode, empty history, and empty provider results have useful recovery paths.
Security	Client only contains public configuration; server secrets remain on Railway; CORS and expensive provider routes have deliberate limits.
Analytics	Track page view, search, title open, Play, source selection, save, rating, error, and resume events without collecting sensitive data.

BOTTOM LINE

Allrated does not need a new stack. It needs a reliable release loop, corrected interaction semantics, calmer loading/error states, and a small set of signature motion and personalization features. Those changes will make the existing product feel substantially more premium.

Source notes: findings are based on the current workspace source, README/replit.md, current preview screenshot, route and schema inspection, and the existing design token/animation files. Recommendations are intentionally implementation-oriented and preserve the current React + Vite + Express + Prisma + Supabase + Railway architecture.